

Prosesslogg Team 20 – 35p

Innholdsfortegnelse

Uke 10	1
Sprint planning 1p.....	1
Workshop – Crazy 8 2p	2
Uke11	5
Daily standup 1p.....	5
Veiledermøte 9.mars 1/4 1p	6
Planning Poker (egendefinert aktivitet) 1p.....	7
Uke 12	10
Sprint review 1p.....	10
Innsiktsarbeid: Intervju 2p	12
Test-Drevet Utvikling (TDD) 2p	13
Uke 13	16
Forelesning med JetBrains (obligatorisk) 1p.....	16
Mobb-programmering (obligatorisk) 2p	17
Demo av MVP/POC (obligatorisk) 2p	19
Uke 14	21
Første retrospektiv (obligatorisk) 2p	21
Arkitekturskisse og dataflyt for MVP 1p	23
Uke 15	25
Veiledermøte 7.april 2/4 1p	25
Identifisering av code smells og refaktorering 3p	26
Backlog Refinement 1p	28
Uke 16	30
Enriching the backlog with quality tags 2p	30
Veiledermøte 13.april 3/4 1p	32
Uke 17	34
Datainnsamling: geriljatesting 2p.....	34
Par-programmering 1p	35
Uke 18	37
Andre retrospektiv (obligatorisk) 2p	37
Datainnsamling: brukertesting (obligatorisk) 2p.....	39
Uke 19	41
Veiledermøte 4.mai 4/4 1p	41
Uke 20	42
Siste veiledermøte 12.mai 5/4	42

Uke 10

Sprint planning 1p

Hvem: Hele teamet

Dato: Onsdag 4.mars

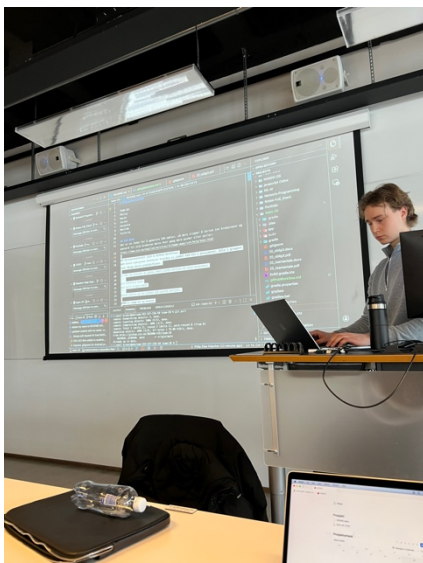
Hvorfor nå?

Teamet valgte å gjennomføre sprint planning i starten av arbeidsperioden for å planlegge utviklingen av UI-skjermene i naturhendelses-appen. På dette tidspunktet hadde vi allerede diskutert appens hovedfunksjonalitet og det var derfor naturlig å planlegge og starte med gjennomføring av koding av brukergrensesnittet. Ved å planlegge sprinten tidlig i prosjektets løp fikk vi en felles forståelse av oppgavene og hvordan arbeidet skulle fordeles.

Hvordan

Vi gjennomgikk backloggens mange oppgaver og identifiserte høyt prioriterte oppgaver knyttet til starten av prosjektets brukergrensesnitt; UI-skjermene: HomeScreen, MapScreen, RiskScreen, SettingsScreen og FavoriteScreen. Oppgavene ble kort diskutert og vi estimerte at en to uker sprint for alle disse ville være nok til å få brukbare, men ikke optimale skjermer. Ansvar ble fordelt blant gruppemedlemmene, men vi hadde en dynamisk tilnærming der vi hjalp der det trengtes.

Bilde/dokumentasjon



Refleksjoner

Sprint planning gjorde det lettere å få oversikt over hva som skulle utvikles og hvem som hadde ansvar for de ulike oppgavene. Aktiviteten bidro til en bedre struktur i arbeidet vårt og reduserte risikoen for at flere jobber i parallell på samme funksjonalitet mye. En utfordring var at visjonen vår ikke samstemte på alle punkter og det krevde lengre diskusjoner. Samtidig opplevde vi at dette var svært nyttig for fremgangen vår og dette var en smidig praksis vi ville fortsette å gjennomføre i prosjektet.

Workshop – Crazy 8 2p

Hvem: Hele teamet

Dato: onsdag 4. mars

Hvorfor nå?

Vi hadde bestemt oss for Case 3 (Naturhendelser), men hadde ellers lite konkret på plass. En løst idé om å gi boliger en klimabasert risikoscore hadde blitt nevnt, men vi var i det store og hele åpne. Vi befant oss i idéfasen og hadde behov for å generere mange ideer raskt og bestemme oss for en målgruppe før vi kunne gå videre med konseptutvikling. Behovet var derfor å utforske mulighetsrommet bredt før vi snevret oss inn på målgruppe og konsept. Crazy 8 egner seg godt nettopp i denne tidlige fasen fordi metoden tvinger frem mange ideer raskt, uten at man låser seg til den første og mest åpenbare løsningen.

Hvordan

Crazy 8 er en tidsbasert idégenereringsmetode fra Design Sprint-metodikken.

Deltagerne bretter et ark i åtte og tegner en ide i hver rute. Målet er å komme seg forbi de åpenbare løsningene og utforske bredden av mulige retninger raskt.

Mål: Generere et bredt spekter av ideer for en app knyttet til Case 3 (Naturhendelser) og lande på en tydelig målgruppe og retning.

Nøkkelspørsmål:

- Hvilken målgruppe skal vi løse et problem for?
- Hvilke naturhendelser er mest relevant å fokusere på?
- Hvordan kan appen se ut, og hvilke funksjoner gir reell verdi for brukeren?

Tidsplan:

Tid	Aktivitet
10 min	Introduksjon og gjennomgang av Case 3
8 min	Crazy 8 – skisser av mulige appskjermer
15 min	Presentasjon av ideer i teamet
15 min	Diskusjon og gruppering av målgrupper og ideer
10 min	Prioritering – valg av målgruppe og konseptelementer
5 min	Oppsummering og veien videre

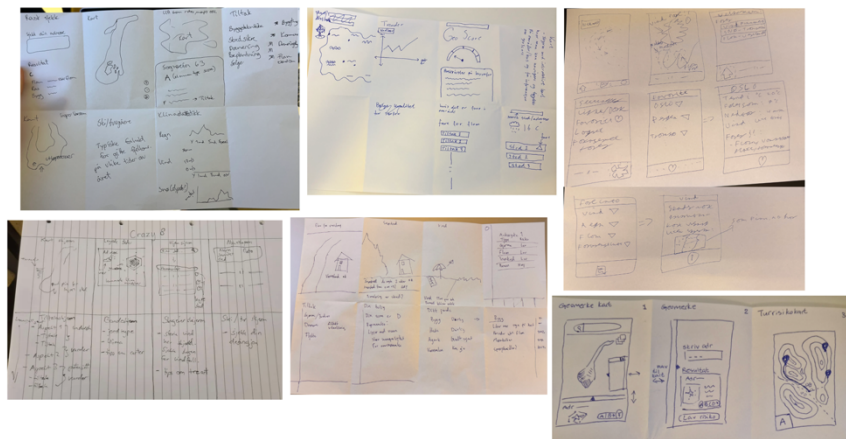
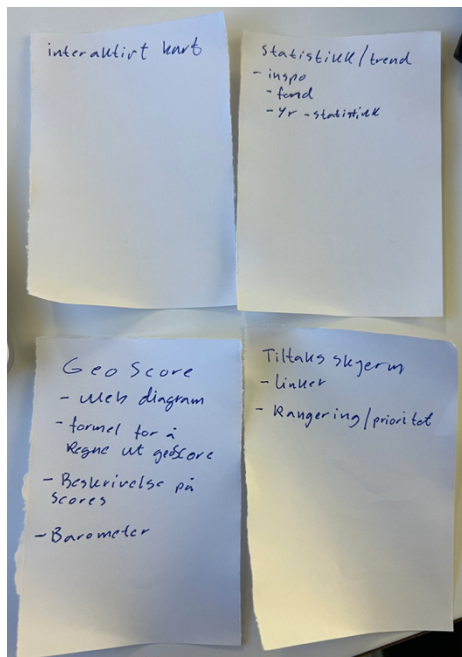
Gjennomføring:

Alle teammedlemmene deltok. Med case 3 som utgangspunkt satt vi på timer og startet med å tegne 8 raske skisser hver med ideer til appen. Dette la til rette for oss til å tenke på brukeropplevelsen tidlig. Vi oppdaget raskt at 60 sekunder er svært kort. Etter 2 skisser utvidet vi tiden til 1 minutt og 30 sekunder. Etter 8 runder holdt vi en presentasjonsrunde der hvert teammedlem presenterte sine skisser og ideer til resten av teamet. Etter presentasjonsrunde diskuterte vi hvilke brukergrupper og funksjoner som gikk igjen på tvers av de forskjellige skissene.

Vi identifiserte tre hovedmålgrupper. Boligeiere/boligutbyggere, skogsbruk og landbruk, og turgåere. Etter diskusjon landet vi på **boligeiere og boligutbyggere som primærmålgruppe**.

Dette er en gruppe caset eksplisitt nevner, der behovet for informasjon om flom- og rasutsatte tomter og konkrete klimatilpasningstiltak er stort. Workshopen ga ideen et konkret grunnlag og utvidet den med flere elementer vi ønsket å ta med videre: interaktivt kart over risikoutsatte områder, geoscore med forklarende detaljrapport, et web diagram, prioriterte anbefalte tiltak med lenker til mer informasjon, samt en side for klimastatistikk med historiske og fremtidige trender.

Bilde/dokumentasjon



Refleksjoner

Positivt: Å skissere konkrete skjermbilder fremfor å bare snakke om ideer gjorde diskusjonen veldig håndfast og produktiv. Metoden fikk alle på banen og skapte felles eierskap til retningen vi valgte. Vi endte opp med en klar målgruppe og konkrete ideer vi alle sto bak.

Negativt: Vi oppdaget at 1 minutt per skisse er veldig kort. Flere av teammedlemmene følte at de ikke rakk å tegne ferdig ideene sine før tiden var ute. Her utvidet vi tiden til 1,5 minutter, men det var et stressmoment, og ikke alle i gruppa satt igjen med 8 skjerm. Kanskje vi gikk glipp av noen verdifulle ideer?

Vil vi gjøre dette igjen? Ja. Crazy 8 er en enkel og effektiv metode tidlig i prosessen.

Uke11

Daily standup 1p

Hvem: Hele teamet

Dato: mandag 9. mars

Hvorfor nå?

Etter forrige ukes crazy 8 workshop hadde teamet flere oppgaver til neste møte. Det ble naturlig å presentere arbeidet som er blitt gjort i en daily standup.

Hvordan

Hvert teammedlem presenterte for resten av teamet arbeidet gjort siden forrige møte. Teamets oppgaver var varierte. Noen nøkkelpunkter var arbeid med prosesslogg, etterarbeid med workshop, navigasjon i appen, google maps vs WMS, API, fargepaletter og UU, rapport og parprogrammering.

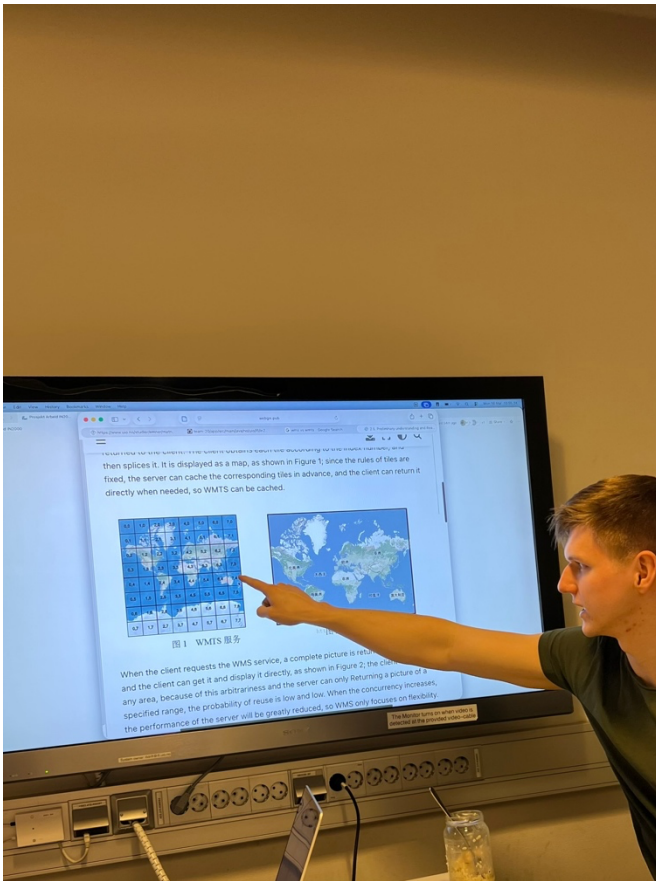
Refleksjoner

Vi erfarte at det var noen ting vi ikke var blitt helt enige om. Det ene teammedlemmet hadde startet arbeidet med API og benyttet HTTP-klienten Retrofit. Etter en kort diskusjon landet teamet på å heller benytte seg av Ktor, da teamet hadde mer erfaring med denne klienten.

Positivt: Standupen tydeliggjorde at det var noen ting teamet hadde glemt å ta stilling til, som valg av HTTP-klient. Dette var en erfaring vi tok med oss. Fremover ville vi være mer oppmerksomme på å diskutere og bli enige om arkitekturvalg i felleskap før arbeidet starter, og dermed unngå unødvendig dobbeltarbeid.

Negativt: Siden dette var vår første standup gikk den over til å bli mer en diskusjon enn en statusoppdatering, og tok dermed lenger tid enn nødvendig.

Bilde/dokumentasjon



Veiledermøte 9.mars 1/4 1p

Hvem: Hele teamet + Veiledere

Dato: mandag 9. mars kl 13:30

Hvorfor nå?

Første veiledermøte etter oppstart av prosjektet. Vi hadde gjennomført Crazy 8-workshopen og trengte tilbakemelding på retningen vi hadde valgt, samt avklaring rundt prosessen og aktiviteter fremover. Teamet valgte å ha ukentlig veiledermøte mandag kl 13:30. Dette er på slutten av vårt ukentlige møte mandag fra 10-14. Tidspunktet ga oss mulighet til å oppdatere hverandre om hva som er skjedd siden sist, planlegge uka og forberede oss til veiledning.

Hvordan?

Vi presenterte valgt case (case 3: naturhendelser) og ideene våre. Etter Crazy 8-workshopen landet vi på primærmålgruppen boligeiere og boligutbyggere. Veilederne ga tilbakemelding og vi diskuterte flere praktiske spørsmål knyttet til gjennomføringen av prosjektet.

Temaer og avklaringer fra møtet:

- Prosessloggen gjelder fra uke 10, men valg som ble gjort i uke 9 som barometer og minigolf kan nevnes i rapporten.
- Avklaring angående poenggivende aktiviteter til prosessloggen.
- Vi bør starte med TDD tidlig, helst allerede neste uke
- Vi går videre med Ktor fremfor Retrofit
- Vi vurderer å gå fra 1-ukes sprinter til 2-ukers sprinter for mer fleksibilitet

Refleksjoner

Møte ga nyttige avklaringer og det var hyggelig å møte veilederne som skal følge oss gjennom prosjektet.

Positivt: Det var nyttig å få tidlige avklaringer rundt både tekniske valg og prosessen rundt prosjektet. Vi fikk bekreftet retningen vi hadde valgt og gikk ut av møtet med en tydeligere prioriteringsliste.

Negativt: Vi hadde forberedt noen konkrete spørsmål til møtet på forhånd, men burde ha satt av mer tid til forberedelser. Diskusjonen ble litt ustrukturert. Til neste veiledermøte ønsket vi å stille med en tydeligere agenda.

Vi ville definitivt gjort dette igjen. Ukentlige veiledermøter var en fast del av prosjektet og noe vi ønsket å fortsette med gjennom hele perioden.

Planning Poker (egendefinert aktivitet) 1p

Hvem: Hele teamet

Dato: fredag 13. mars

Hvorfor nå?

På mandag gjennomførte teamet en mobbprogrammering. Denne var ikke grundig nok planlagt på forhånd og gikk ikke veldig bra. Teamet støttet på tekniske problemer med gradle-filer og feil prosjektkonfigurasjon. 5-minutters-rotasjonen passet dårlig til den typen oppgaver vi hadde. Vi fullførte hele timen med bytting, men satt igjen med følelsen om at dette fikk vi svært lite ut av. Som et team bestemte vi oss for å skrote denne mobbprogrammeringen og heller gjøre den senere. Etter å ha sett på alternative aktiviteter landet vi på Planning Poker. Dette passet bra siden vi uansett hadde behov for å estimere MVP-backloggen før møtet på mandag og frem mot demoen i uke 14.

Hvordan

Vi benyttet Planning Poker, en smidig estimeringsteknikk der hvert teammedlem uavhengig tildler storypoints til backlog-oppgaver. Skalaen vi brukte var 1, 2, 3, 5, 8 og 13. 1 er en svært liten oppgave estimert til å ta opp til 2 timer med arbeid og 13 indikerer at den vil ta mer enn en uke, har mange dependencies og i grunn bør splittes opp. Alle gjorde hvert sitt estimat og fylte de inn i Notion. Teamet gikk så igjennom estimatene i fellesskap.

Refleksjoner

Positivt: Det var hjelpsomt å gå gjennom backloggen. Teamet stort sett samkjørte i estimatene sine, men det var noen av de som spriket.

Negativt: Teamet har forskjellige bakgrunner, og varierende grad av erfaring med liknende arbeid. Dette førte til at noen av oppgavene var mer krevende enn andre å estimere, da man ikke nødvendigvis har et innblikk i hva hver av dem innebærer.

Det ble også fristende å søke etter andres tanker om arbeidsoppgaven via kommentarer om de. Oppgaven API-1 «*Lage struktur for lokasjonssøk i ViewModel og states*» hadde kommentaren «*Sentral logikk, relativt krevende*». Det er ikke utenkelig at estimatene 8/5/5/8/5/? ble påvirket av dette.

Bilde/dokumentasjon



D. MapScreen — MÅ med til MVP

ID	Oppgave	Story points	Kommentar
MAP-1	Lage grunnlayout for MapScreen	3/2/2/5/3/?	Overordnet skjerm (nærmer oss ferdige allerede)

API-1	Lage struktur for lokasjonssøk i ViewModel og states	8/5/5/8/5/?	Sentral logikk, relativt krevende
-------	--	-------------	-----------------------------------

C. Search / Location — MÅ med til MVP



ID	Oppgave	Story points	Kommentar
SEARCH-1	Lage grunnlayout for SearchScreen	8/8/8/8/3/?	Struktur på skjermen (ref. Victorias skjerm)

Uke 12

Sprint review 1p

Hvem: Hele teamet

Dato: Mandag 16.mars

Hvorfor nå?

Teamet gjennomførte sprint review i uke 12 fordi vi da hadde fullført den oppsatte 2 ukers sprinten vi laget i sprint planning aktiviteten i uke 10. Sprinten hadde fokus på UI-skjermer og navigasjon i naturhendelses-appen. På dette tidspunktet hadde vi nok ferdig arbeid til å kunne evaluere hva som faktisk var blitt utviklet. Samtidig hadde vi nok tid til å vurdere endringer og forbedringer videre inn mot MVP-demo fristen om 2 uker. Grunnet dette ble sprint review gjennomført nå, der sprinten er friskt i minnet og forbedringer er enklere og nyttigere å implementere for den siste sprinten frem til MVP-demo i uke 13.

Hvordan

Vi gikk sammen målet for sprinten og sammenlignet dette med det vi faktisk fikk utviklet. Vurderingsgrunnlaget ble lagt på UI-skjermene, navigasjonen mellom skjermene og hvor langt på vei til en ferdig MVP vi hadde kommet. Vi diskuterte hindringer som blokkerte fremgang, utvidelser og nye ideér. Til slutt oppsummerte vi hva som var oppnådd, hva som fortsatt manglet og hvilke justeringer som burde gjøres i backloggen og prosessen videre.

Refleksjoner

Aktiviteten sprint review gjorde det lettere å få en felles forståelse av hvor langt vi faktisk hadde kommet i prosjektet. En positiv erfaring er at vi kan se tilbake på sprinmålene og se at vi faktisk hadde oppnådd mer enn målet vi hadde satt, men at det også hadde kommet mange flere interessante oppgaver. Navigasjonen hadde blitt satt opp med Navigation 3 som er nyere og veldig enkelt å forholde seg til, videreutvikle og forstå.

På den andre siden, så ser vi at vi hadde for brede sprint mål og backlog oppgaver, noe som gjorde målet og resultatet vanskelig å konkludere. Noe av grunnen til dette handler om at oppgavens skop vokste underveis, der UI-designet kontinuerlig utvikler seg og utviklerne sliter med å fullføre alle de gode ideéne.

Denne aktiviteten var svært nyttig, og det er noe vi ønsket å fortsette med å gjennomføre som et tverrfaglig og dynamisk team.

Bilde/dokumentasjon

Sprint review

1. **Hva vi fikk til**
HomeScreen, MapScreen, RiskScreen, SettingsScreen og FavoriteScreen
navigasjon mellom skjermene
konkretisert hva vi ønsker av funksjonalitet
lett kan videreutvikle med dette utgangspunktet
2. **Hva synes vi?**
bra - fikk gjort alle skjermene vi planla for
godt skjelett som er fint å jobbe videre med
bra overordnet
arkitekturen forankret i "best practice"
godt utgangspunkt
basert på mye antakelser om brukerne våre
3. **Videre**
FavoriteScreen mangler bottom nav bar
vi trenger en searchScreen
trenger brukerinnsikt
dele opp oppgaver i mindre deler

Refleksjon til sprint review

- fint at vi fikk tatt opp at sprinten går for fort

Innsiktsarbeid: Intervju 2p

Hvem: Veronica

Dato: 18.mars, 2.april og 3.april

Hvorfor nå?

Teamet valgte å gjøre innsiktsarbeid på dette tidspunktet, da vi hadde etablert såpass mye av ideen vår gjennom workshop med crazy 8 og diskutert detaljer der vi fikk flere og flere spørsmål å ta stilling til, både relatert til appen og domenet. Her kjente vi behovet for å la brukere gi deres perspektiver og innsikt. Ettersom man er avhengig av andres tid og tilgjengelighet, ønsket vi å starte prosessen tidlig. Første intervjuet ble i uke 12, men siden vi ønsket innsikt fra flere brukere, og deres tilgjengelighet, ble de to andre i uke 14.

Hvordan

Metoden falt på intervju med presentasjon av ulike prototyper. Vi intervjuet totalt fire brukere. Intervjuene ble en semistrukturert variant med stikkord og spørsmål, men der noen ble hoppet over og andre nye spørsmål oppstod. Som supplement brukte vi lavoppløselige prototyper som kommunikasjonsverktøy.

Refleksjoner

Vi er erfarte at brukere er ulike. Vi opplevde nyttigheten med prototyper som kommunikasjonsverktøy. Det var i tillegg motiverende med tilbakemelding på at ideen er god. Brukernes ulike meninger gjorde det dog litt vanskeligere å vite hva man skulle gå videre med, derfor kunne vi ønsket å ha data fra enda flere brukere for å være mindre basert på enkeltindividers subjektive meninger. Vi tenkte derfor at å ha spørreskjema kunne vært et nyttig supplement.

Vi ville definitivt gjøre dette igjen senere i prosessen som en viktig del av utviklingen. Det var flere designvalg å ta der brukerne burde involveres for å få innsikt i deres perspektiv, avdekket deres behov, og teste hvorvidt det vi har utviklet fungerer som tenkt.

Bilde/dokumentasjon



Test-Drevet Utvikling (TDD) 2p

Hvem: David, Jurius og Matilda

Dato: fredag 20.mars

Hvorfor nå?

Teamet valgte å gjennomføre TDD på dette tidspunktet da vi anså det som hensiktsmessig å gjennomføre tidlig i prosjektet. Dette var riktig nok det andre forsøket. Den første gangen beregnet vi for lite tid til aktiviteten. Vi planla å fortsette på neste møte, men en tydeliggjøring av casebeskrivelse var kommet inn i mellomtiden. Det var ikke lenger aktuelt å bruke MetAlerts APIet vi startet TDD-en med, da den ikke lenger passet inn i caset. Teamet ble enige om at å gjøre TDD for et API vi ikke lenger skal benytte oss av gikk imot hensikten til TDD og det ble bestemt å starte på nytt.

Hvordan

For at teamet skulle rekke å bli klare til MVP ble vi enige om å splitte oss opp. 3 av medlemmene møttes på fredag 20. mars for å gjennomføre aktiviteten. Denne gangen med Frost v0 API-et som vi vil bruke i appen.

Poenget med TDD er å skrive tester før man skriver koden som utføres. Dette tvinger oss til å tenke over hva det er vi ønsker at hver metode skal returnere. TDD er en iterativ prosess med stegene Red-Green-Refactor. Først skriver man en test som skal feile, deretter minimal kode for å bestå testen. Når testen er bestått er fokuset på refektorering, opprydding, forbedring av kodens struktur, lesbarhet og ytelse uten å endre på adferden.

Vi møttes og jobbet oss gjennom en funksjonalitet per lag, API, Repository og ViewModel. Alle testene ble skrevet med JUnit, det innebygde testrammeverket i android-prosjekter. Det håndterer oppsett, kjøring og rapportering av testene. For å slippe ekstra avhengigheter laget vi egne fake-klasser i stedet for å bruke et mock-bibliotek. Disse implementerer de samme interfacene som den ekte koden og lar oss teste hvert lag uavhengig av hverandre.

Vi startet med å skrive testene, kjørte så gjennom de for å forsikre oss om at de feilet. Deretter implementerte vi koden og fikk de til å passere, før vi til slutt ryddet opp i struktur og navngivning.

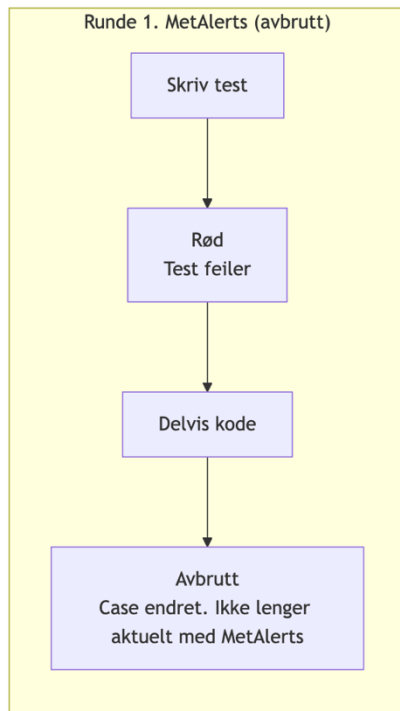
Refleksjoner

Positivt: Ved å skrive testen først måtte vi mer bevisst tenke gjennom hva koden skal gjøre. Dette lot oss avdekke flere ting vi ikke hadde tenkt på, særlig rundt feilhåndtering i ViewModel. Fake-klassene var også overraskende enkle å lage og gjorde testene mer lesbare.

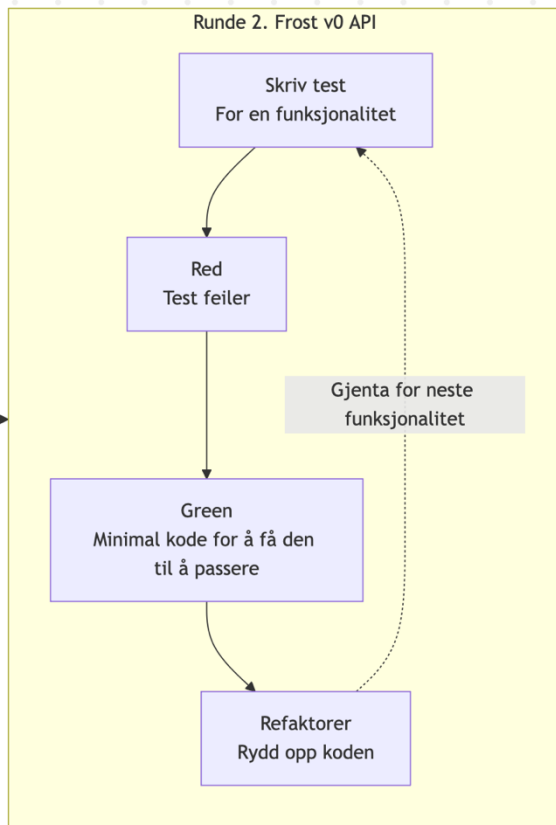
Negativt: Det var litt knotete å komme i gang. Vi brukte en del tid på å få ViewModel-testene til å fungere med coroutines. `viewModelScope.launch` var nemlig koblet til hovedtråden, `Dispatchers.Main`. Denne tråden eksisterer ikke i JUnit testmiljøet, så testene krasjet. Etter mye frem og tilbake løste vi til slutt dette ved å lage en `MainDispatcherRule` som byttet ut hovedtråden med en test-versjon før hver test kjører.

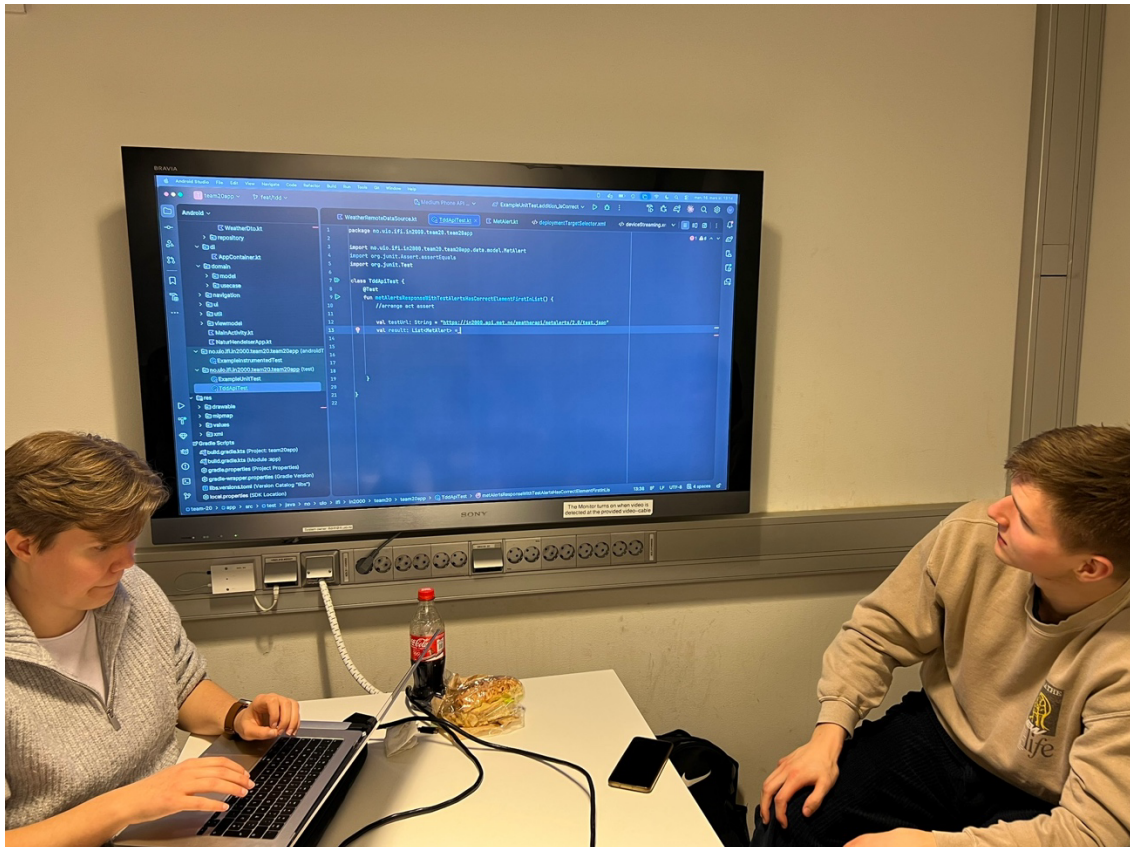
Vil vi gjøre dette igjen? Ja. Vi følte vi fikk god nytte av TDD, spesielt for å stoppe opp og tenke over hva det egentlig er vi skal teste, og dermed implementere. Teamet tok med seg erfaringene videre i prosjektet.

Bilde/dokumentasjon



Start på nytt →





Uke 13

Forelesning med JetBrains (obligatorisk) 1p

Hvem: Hele teamet

Dato: 25. mars

Hva

Obligatorisk forelesning med JetBrains ble gjennomført 25. mars. En ansatt i JetBrains fra Nederland fortalte om structured concurrency i Kotlin.

Refleksjoner

Positivt: Det var litt stas å ha forelesning fra en av de som er med å lage Kotlin. Forelesningen gjorde også at vi fikk øynene opp for at det var mye mer å lære om concurrency i Kotlin og inspirerte egen lesing og undersøkning i etterkant.

Negativt: Gjesteforeleserens strategi av å kaste produkter rundt om i salen for å drive publikumsinteraksjon ble litt forstyrrende for noen av oss i lengden (David fikk en plushie i hvert fall). Forelesningen var tettpakket med mye informasjon. Teamet skulle derfor gjerne hatt muligheten til å se forelesningens innhold igjen i etterkant. Dette tenker vi ville bidratt til å få mer ut av aktiviteten.

Hvordan vi vil anvende dette i vårt prosjekt:

Vi anvender structured concurrency til å gjøre API-kall parallelt. Vi ønsker ikke error propagation, i grensetilfellet av delvis data ønsker vi fremdeles resultatet, så vi håndterer errors der de oppstår.

Bilde/dokumentasjon



Mobb-programmering (obligatorisk) 2p

Hvem: Hele teamet

Dato: 26. mars

Hvorfor nå?

Teamet hadde bestemt seg for å gå bort fra LocationForecast og å rydde opp i hvordan lokasjon ble håndtert på tvers av appen. Dette var en omfattende refaktoreringsjobb som berørte mange filer samtidig. Vi vurderte det som en god oppgave for mobb-programmering siden alle i temaet burde ha en god forståelse av endringene

Hvordan

Vi la en plan for økten

1. Fjerne LocationForecast
2. Oppdatere HomeViewModel som brukte LocationForecast

3. Oppdatere AppViewModel til å bruke Location-dataklassen i stedet for CreatedLocation
4. Vise lagrede lokasjoner i en LazyColumn

Vi koblet til en pc på stor skjerm, og fulgte anbefalingene fra forelesningen med bytte hvert 5. minutt. Et teammedlem skrev kode mens resten av teamet satt rundt i en sirkel og dikterte og diskuterte. Hver person fikk kode i 2 bolker på 5 minutter hver.

Endringene vi gjorde gikk i hovedsak ut på å erstatte CreatedLocation med en ny felles Location-dataklasse, og sende denne videre gjennom navigasjon, ViewModels og skjermer i stedet for å sende name, lat og lon som separate parametere. Dette ryddet opp en del støy i koden og gjorde det enklere å utvide Location med mer informasjon senere. Tiden gikk fort og vi rakk ikke å starte på punkt 4.

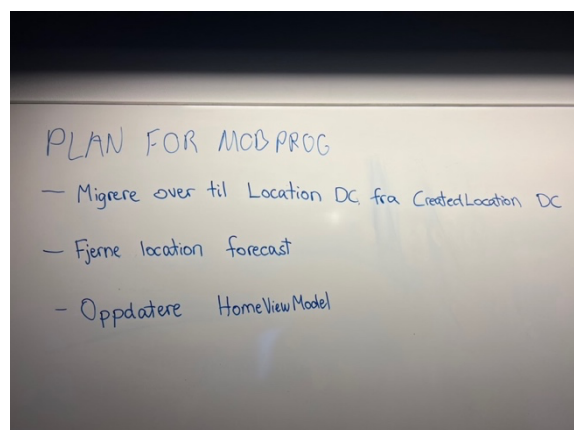
Refleksjoner

Positivt: Det var nyttig at hele teamet var til stedet under en større refaktorering. Dette gjorde at alle fikk en forståelse av hvordan lokasjon flyter gjennom appen. Vi opplevde også at det ble lettere å stille spørsmål underveis når hele teamet var samlet.

Negativt: Vi fikk litt opplevelsen av at flere kokker kan føre til mer søl. 5 minutter er kort, spesielt når man er midt i en endring som berører mange filer. Det ble mange kontekstbytter, og noen uklare beskjeder. Vi kom etter hvert inn i en flyt hvor vi i stor grad benyttet oss de nummererte kodelinjene for å tydeligere kommunisere til den som satt og kodet.

Vil vi gjøre dette igjen? Det var en fin og lærerik opplevelse å ha, men vi ville trolig vært mer effektive i mindre grupper. Mobprogrammering fungerte greit for refaktoreringen, men vi er litt usikre på om det er riktig bruk av hele teamets tid fremover.

Bilde/dokumentasjon



Lenke til mobb-programmerings comitten:

<https://github.uio.no/IN2000-V26/team-20/pull/24/commits/2add6177a97265be69034a957973da243e2c1276>

Lenke til PR fra mobbprogrammeringen:

<https://github.uio.no/IN2000-V26/team-20/pull/24>

Demo av MVP/POC (obligatorisk) 2p

Hvem: Hele teamet + veiledere

Dato: 27. mars

Vi viste frem MVP-en vår til veilederne fredag før påskeuka. Appen inneholdt en hjemskjerm, søk, kart, lagrede adresser og innstillinger. Fra hjemskjermen kunne brukeren søke opp en adresse og få hentet ut gjennomsnittlig temperatur og nedbørsmengde fra nærmeste værstasjon, de tre siste månedene. Appen hentet dataen fra Frost v0 og viste det grafisk i en enkel tabell. Vi fikk gode tilbakemeldinger fra veilederne flere ideer om videre implementasjon av

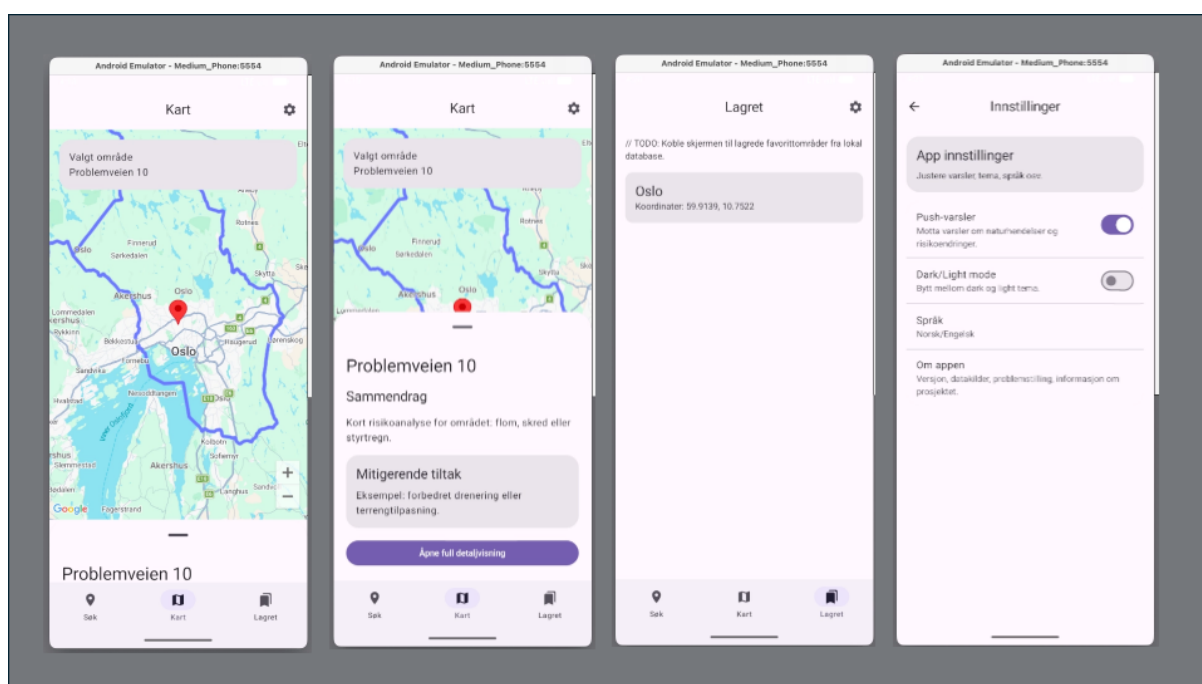
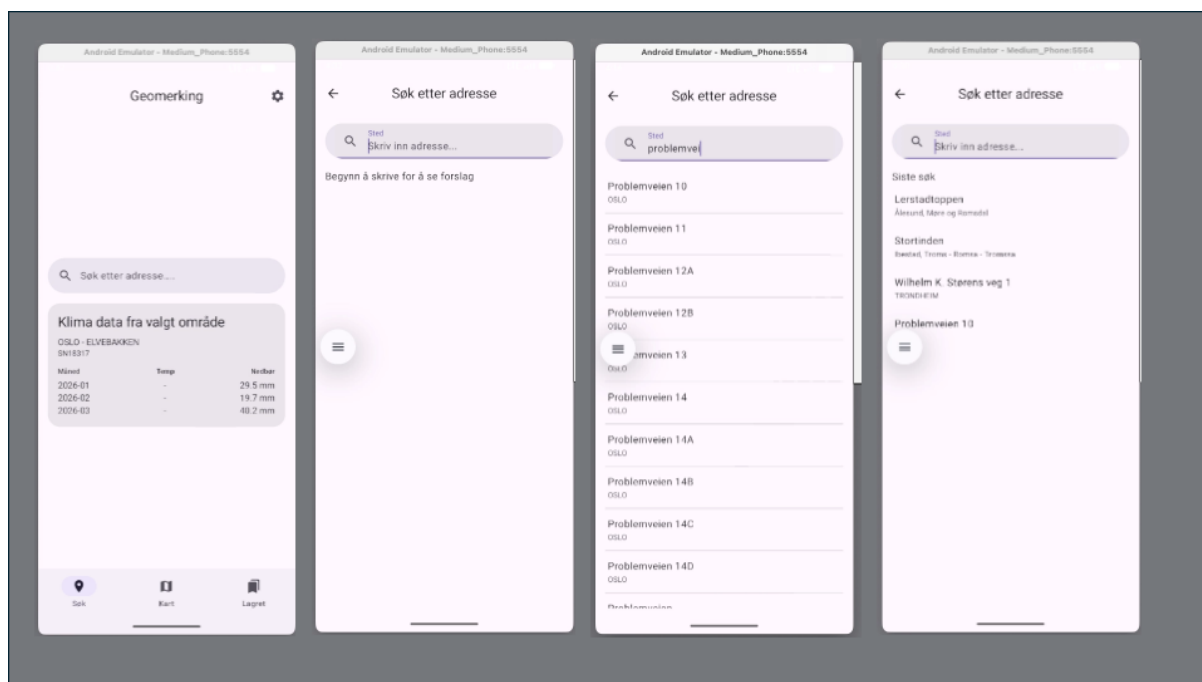
Refleksjoner

Positivt: Det var motiverende å vise frem noe konkret og få positive tilbakemeldinger på der vi hadde fått til så langt.

Negativt: Påsken kom forttere enn forventet. Vi skulle gjerne vist frem en MVP som i større grad reflekterte vår vinkling av oppgaven, med risikovurdering og visuell representasjon av historisk klimadata, men det rakk vi ikke. Vi tar med oss tilbakemeldingene og lærdommene videre i prosjektet.

Bilde/dokumentasjon

Lenke til siste commit for MVP: <https://github.uio.no/IN2000-V26/team-20/commit/d89723403950ebc67311484ad1e6ae51d01a948b>



Uke 14

Første retrospektiv (obligatorisk) 2p

Hvem: Hele teamet

Dato: 30.mars

Hvorfor nå?

En måned inn i prosjektet var det passende å ta en retrospektiv i fellesskap med hele teamet til stede. Det kjentes ut som tiden var moden da vi hadde fått tid til å bli kjent, se hverandre i arbeid og kjent på hvordan samarbeidet så langt hadde gått. Dette var fint i det vi skulle besvare spørsmålene stilt i aktiviteten. Samtidig var det såpass tidlig i prosjektet slik at vi kunne ta med oss avdekkede forbedringspunkter i videre arbeid.

Hvordan

For gjennomføring av aktiviteten brukte vi et verktøy vi fant på nett kalt Echometer. Her var det samlet flere Templates vi kunne velge mellom. Vi valgte å bruke «Health Check Items», «Good/Bad», «Loved, Learned, Longed», «Hot Air Balloon Retro», «Lean Coffee» og «Keep, Stop, Start». For å effektivisere samt la team-medlemmene tenke gjennom spørsmålene alene i eget tempo, opprettet vi en anonym survey med de tre førstnevnte templatesene. Dette ble så diskutert i plenum i tillegg til de resterende templatesene. For både å bli kjent med verktøyet og som oppvarming til aktiviteten, startet retroen med en «icebreaker». Vi satte av 1t til aktiviteten i plenum, som også viste seg å være nødvendig da en full time ble brukt.

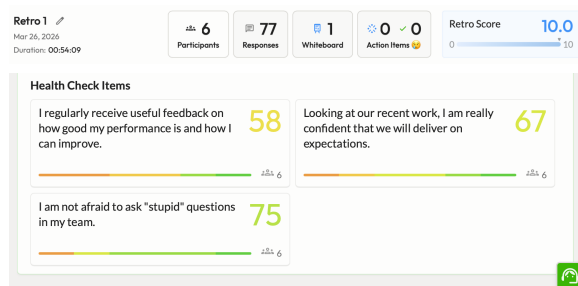
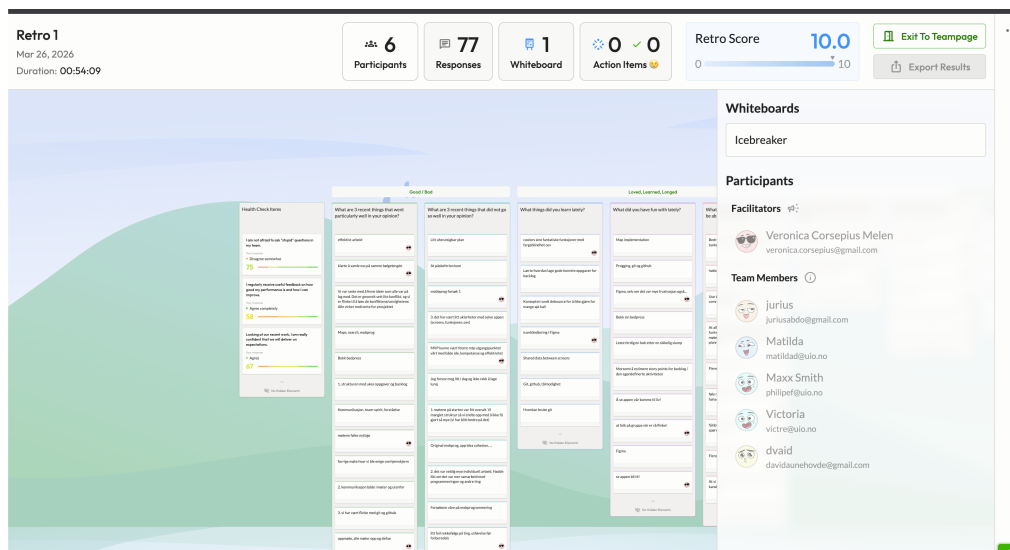
Refleksjoner

På scoren på slutten av retroen skåret teamet anonymt aktiviteten som 10/10. Vi erfarte dette som en fin anledning til å ta opp blockers for godt samarbeid, forstyrrende elementer for teamdynamikken, men også kunne sette pris på og verdsette positive sider ved samarbeidet. Aktiviteten skapte rom for en åpen og ærlig samtale. Vi erfarte at det ble i overkant mange templates. Det var absolutt noe vi kunne tenkt oss igjen, men planla for å redusere antall templates til neste retro.

Keep, Stop, Start

Vi ble enige om å fortsette det gode engasjementet, gi hverandre positiv feedback på GitHub, gode commits osv., og å ha fast møte på mandager. Som forbedringspunkter ble vi enige om å legge inn et møte til hver uke med statusoppdatering og arbeide mer sammen. Vi diskuterte ønske om mer struktur, med tanke på møtene, arbeidsoppgavene og tydeligere avgrensede sprints, for å bli mer samkjørte. Vi kjente også behov for bedre alignment på hva appen skulle være, da vi oppdaget at vi manglet litt felles forståelse.

Bilde/dokumentasjon



Arkitekturskisse og dataflyt for MVP 1p

Hvem: Jurius, David, Matilda

Dato: 30.mars

Hvorfor nå?

Forklarer kort hvorfor teamet valgte å gjennomføre aktiviteten på dette tidspunktet (og ikke tidligere/senere i prosjektarbeidet).

Vi valgte å utarbeide arkitekturskissen uken etter MVP-innleveringen, ettersom vi på det tidspunktet hadde landet en stabil apparkitektur som vi ønsket å ta med oss videre i prosjektet. Dette gjorde at skissen i stor grad også reflekterer arkitekturen i det endelige produktet.

Hvordan

Tre av gruppemedlemmene med størst ansvar for apparkitektur og implementering gjennomførte en digital samtale i påskeferien for å gå gjennom og diskutere den nåværende strukturen i prosjektet. Jurius samlet deretter informasjonen fra samtalen og utarbeidet et flytdiagram i Mermaid ut ifra den. Etter samtale med veileder ble det gjort noen justeringer, da den originale skissen lignet mer et dataflytdiagram.

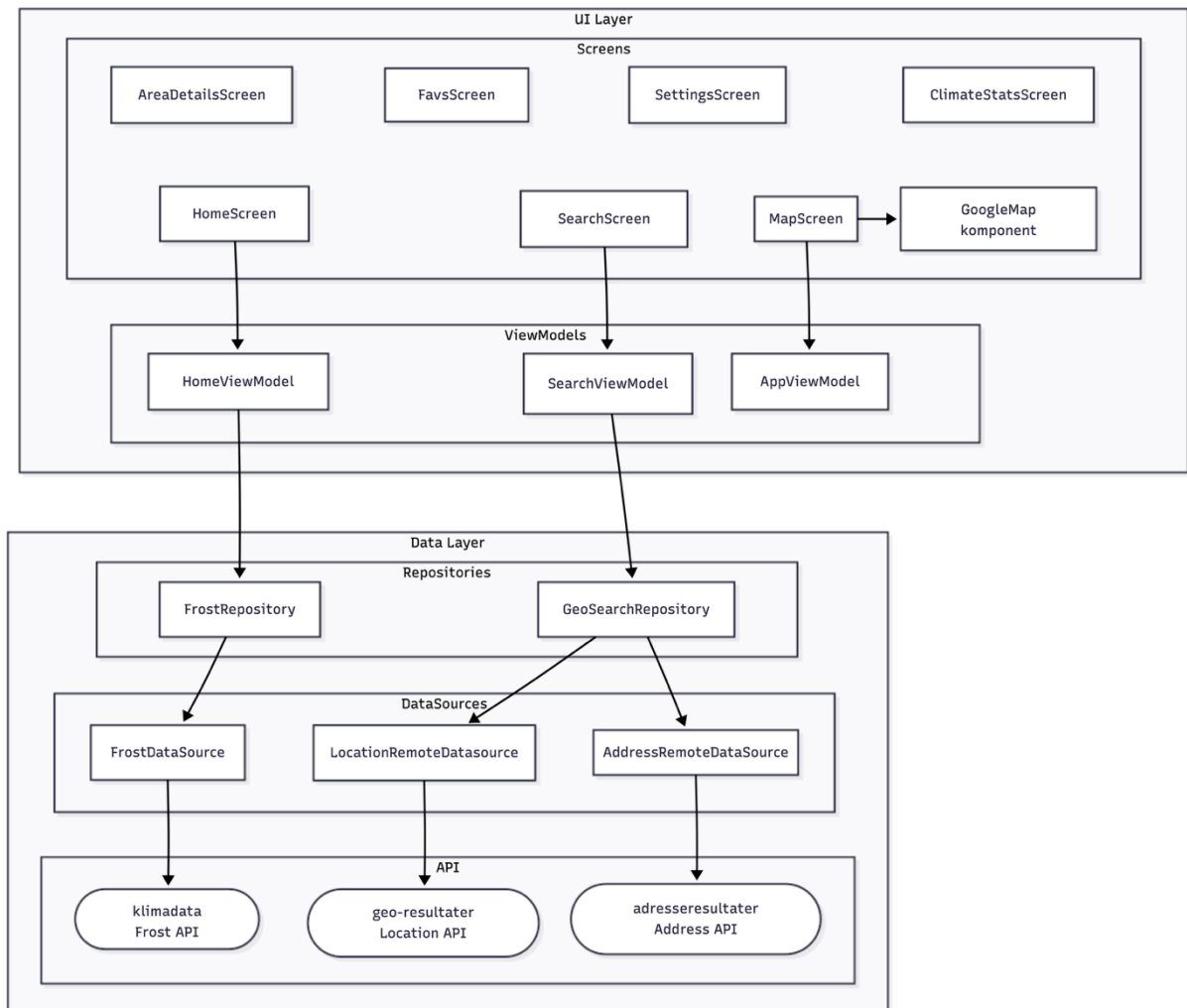
Refleksjoner

Vi syns at å lage en arkitekturskisse var hjelpsomt på mange forskjellige måter. For eksempel fikk alle i gruppen (til og med de som ikke hadde ansvar innenfor app arkitektur og programmering) nå en overordnet forståelse i hvordan appen er bygd opp og hvordan dataen flyter under panseret. Arkitekturskissen satt søkelys på mange unøyaktigheter i koden også som vi endte opp med å fikse. Alt i alt var det hjelpsomt å visualisere app strukturen, og det er nok noe vi vil ta med oss videre i andre deler av prosjektet.

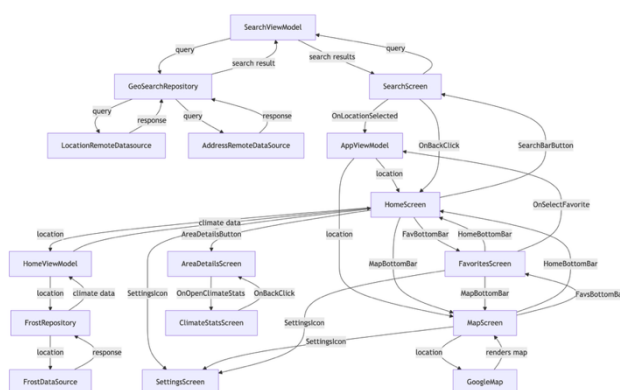
I tillegg avdekket prosessen flere unøyaktigheter i koden som vi ikke hadde lagt merke til tidligere, og som vi i etterkant fikk rettet opp. Det å visualisere systemet tvang oss til å tenke presist rundt ansvar og dataflyt på en måte som ren koding ikke alltid gjør.

Alt i alt var dette en arbeidsmetode vi ønsket å ta med oss videre, og vi så for oss at lignende visualiseringsøvelser kan være nyttige også i andre deler av prosjektarbeidet fremover.

Bilde/dokumentasjon



Dataflyten kan også ses i arkitekturskissen vi først tegnet:



Uke 15

Veiledermøte 7.april 2/4 1p

Hvem: Hele teamet minus Veronica og Philip

Dato: 7. april

Hvordan:

Etter påsken ble del 2 av teamaktivitetene publisert. Før veiledermøtet la vi en plan for hvilke aktiviteter vi skulle gjennomføre når.

Temaer og avklaringer knyttet til møtet:

- Viste frem arkitekturskissen. Noen mindre ting som bør fikses, f eks så bør pilene for dataflyt gå begge veier
- Pass på at appen ikke krasjer uten nett. Tydeliggjør gjerne i staten at det er internett det er et problem med.
- Vi kunne forsøke å bruke Model Context Protocol (MCP) i Figma for å få kode fra Figma og inn i appen.
- Huske å teste appen i forskjellige skjermstørrelser. Gjerne også nettbrett.
- Produktkrav, produktkrav, produktkrav!
- Etter påsken var det blitt publisert nye teamaktiviteter. Vi presenterte planen for gjennomførelsen av disse til veileder. Dette var svært nyttig, og vi fikk flere nyttige innspill. I utgangspunktet hadde vi planlagt at hele teamet skulle delta på begge ukas aktiviteter, identifisering av code smells og refaktorering og Backlog Refinement. Vi ble rådet av veileder til å splitte oss opp. Code smells passet visstnok best for en litt mindre gruppe.

Refleksjoner:

Første veiledermøte etter påske fungerte som en fin avklaring av aktivitetene for resten av prosjektet. Vi fikk flere nyttige tips angående aktivitetene, Figma, skjermstørrelser og påminnelser om å ikke glemme produktkravene.

Identifisering av code smells og refaktorering 3p

Hvem: Jurius, David og Victoria

Dato: 9. April

Forklarer kort hvorfor teamet valgte å gjennomføre aktiviteten på dette tidspunktet (og ikke tidligere/senere i prosjektarbeidet):

Vi valgte å gjennomføre refaktoreringen på dette tidspunktet fordi kodebasen hadde vokst uten at vi hadde gjort noen systematisk opprydding underveis. Ved å identifisere og håndtere code smells relativt tidlig i prosjektet ønsket vi å etablere en ryddig struktur før vi bygget videre. Tanken var at det ville være enklere og mindre tidkrevende å rydde opp nå enn å la teknisk gjeld bygge seg opp og bli et større problem senere.

Som forarbeid gikk ett av medlemmene grundig gjennom oppgaveteksten for å kunne presentere den på en tydelig og effektiv måte for resten av gruppen. Vedkommende noterte også ned forslag til filer som inneholdt code smells og som kunne være aktuelle kandidater for refaktorering. Ett av disse forslagene dannet grunnlaget for selve aktiviteten.

Gruppen ble enige om å refaktorere SearchViewModel. Før vi begynte med å identifisere og refaktorere code smells, skrev vi tester for SearchViewModel. Disse testene brukte vi underveis for å verifisere at refaktoreringene ikke endret funksjonaliteten.

Vi startet med å identifisere code smells uten noen verktøy. Etter en grundig gjennomgang av filen fant vi blant annet lange funksjoner med flere ansvarsområder, unøyaktige kommentarer og unødvendig kode. Selve refaktoreringen gjennomførte vi i plenum, ettersom vi så verdien av å kunne diskutere løsninger fra flere synsvinkler. Den egne refaktoreringen tok rundt 1,5 timer, noe som var mer enn vi hadde planlagt.

Vi valgte å delegere UiO GPT-delen til ett enkelt medlem slik at vi kunne arbeide parallelt, og fordi vi antok at denne delen ville være mindre krevende enn den manuelle refaktoreringen. UiO GPT hadde imidlertid noen begrensninger når det gjaldt identifisering av code smells. Den fanget ikke opp de mest sentrale problemene vi hadde funnet i vår egen gjennomgang. Vi måtte derfor gi hint for å lede den i riktig retning. Selve refaktoreringen klarte den derimot å gjennomføre på én iterasjon.

Mot slutten svarte vi på skjemaet i plenum for å få meningene til alle sammen med.

Refleksjoner:

Aktiviteten ga oss verdifull innsikt, både når det gjelder arbeidsprosesser og tilstanden til kodebasen vår. Under identifiseringen av code smells oppdaget vi blant annet at vi ikke benyttet dependency injection i flere deler av koden. Fordi vi gjennomførte aktiviteten relativt tidlig i prosjektet, kunne vi nå rette opp i dette før det utviklet seg til et større problem.

Vi fikk også erfaring med å bruke verktøy som UiO GPT som en del av utviklingsprosessen. Basert på våre erfaringer fungerer UiO GPT best til selve kodingen og implementeringen, men er mindre sterk på analyse og identifisering av problemer. Dette er en nyttig innsikt vi kunne ta med oss videre i prosjektet: vi kan bruke mer tid på grundig analyse og planlegging selv, og til en viss grad la verktøy som UiO GPT bistå med selve implementeringen.

Aktiviteten var lærerik og ga oss et nytt perspektiv på kodebasen. Vi kommer likevel nok ikke til å gjennomføre den på nøyaktig samme måte i fremtiden. Det vi har lært vil vi absolutt ta med oss videre, og vi vil fortsette å identifisere og refaktorere code smells løpende. Selve oppsettet fra teamaktiviteten var imidlertid ikke den mest effektive tilnærmingen for oss.

Bilde/dokumentasjon



(commit historien for Own branchen)

```
667d41e (HEAD -> own_refactoring, origin/own_refactoring) renamed SearchViewModel to OWN_SearchViewModel
49b1230 added unit tests from before_refactoring branch to own_refactoring branch
8351fac removed some unused imports in searchViewModelUnitTest
983e579 added unit tests for searchViewModel
cf15f04 Made unit test file for searchViewModel
0f2cbe0 Made SearchViewModels datasource constructor injected so that unit tests can more easily be written.
487d4c3 Translated and clarified comments
6071aff Removed redundant address validation and made a helper function for getting search results, sendQueryToRepo'
```

(commit historien for GPT branchen)

```
commit ecc2e5ec0981880aec13189b1e3faeabf75a588f (HEAD -> after_refactoring_uio_gpt, origin/after_refactoring_uio_gpt)
Author: jeryosa <jeryosa@ulrik.uio.no>
Date: Fri Apr 10 16:57:26 2026 +0200

    renamed searchviewModel file to uio_GPT_SearchViewModel

commit e89a8dfb5c5b02378848003e9657e5734dd466aa
Author: jeryosa <jeryosa@ulrik.uio.no>
Date: Fri Apr 10 16:53:10 2026 +0200

    refactored seachviewModel with uiogpt by:
    1. moving query validation into searchViewModel
    2. added helper funksjons for state update
    3. removing unnecessary comments and translating comments we need to english
    4. splitting updateinput into updateinput and preformsearch
    5. removing unused imports

commit ddbfb07f7ea1bb2e3e93e68f42a1ba4991087799 (origin/before_refactoring, before_refactoring)
Author: jeryosa <jeryosa@ulrik.uio.no>
Date: Fri Apr 10 11:42:37 2026 +0200

    removed some unused imports in searchViewModelUnitTest

:

    renamed searchviewModel file to uio_GPT_SearchViewModel
```

Backlog Refinement 1p

Hvem: Matilda, Veronica, Victoria og Philip

Dato: 10. april

Etter påsken hadde vi behov for en konkretisering og en ny gjennomgang av backloggen fra planning poker.

Hvordan:

Som forarbeid til aktiviteten gikk et av teammedlemmene nøye gjennom casebeskrivelsen og produktkravene og utarbeidet to forskjellige backlogger. Og formulerte noen brukerhistorier. Disse fungerte som et utgangspunkt for aktiviteten.

Under aktiviteten gikk vi gjennom brukerhistoriene og hele backloggen. Det ble gjort noen mindre endringer og vi la til og delte opp et par av issueene i mindre issueer.

For å estimere og prioritere oppgavene gikk vi igjen gjennom hver av oppgavene, og startet med å prioritere de etter høy, middels og lav viktighet. Dette var ikke veldig lett, nesten alle ble satt til høy, da de stammer fra obligatoriske krav i casebeskrivelsen og produktkravene. Vi gikk over til å prioritere basert på hva som må gjøres først, senere, sist og hva som må gjøres fortløpende. Denne prioriteringen gjorde vi med bruk av post-it lapper på tavla. Noe som kunne minne om dependency graf?

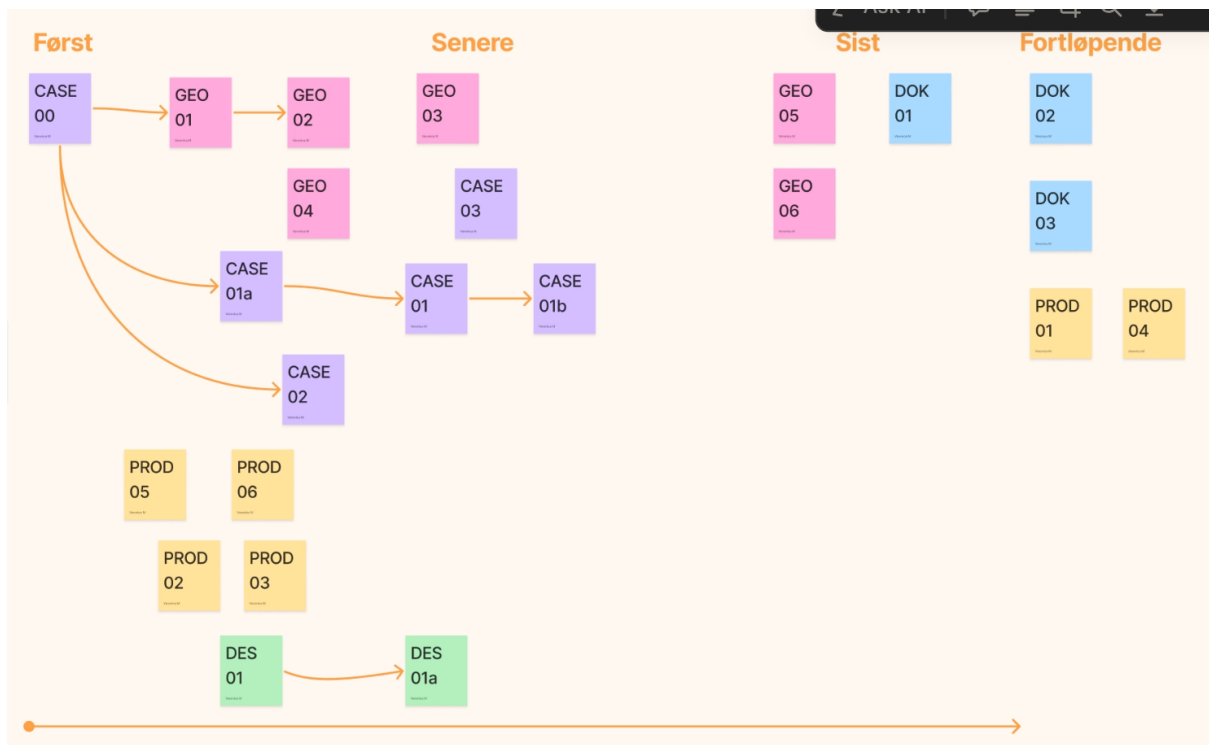
Her ble det tydelig at 3 av de større tingene kan gjennomføres i parallell, da de egentlig ikke har noe med hverandre å gjøre.

Refleksjoner:

Positivt: Veldig, veldig nyttig!

Bilde/dokumentasjon





Uke 16

Enriching the backlog with quality tags 2p

Hvem: Veronica, Victoria, Matilda og Julius

Dato: 13. april

Etter forrige ukes Backlog Refinement hadde vi en godt strukturert backlog som teamet var kjent med. Dette ble et naturlig tidspunkt for å enriche den videre med kvalitetstagger, siden vi hadde nye issues å jobbe med og et felles bilde av hva de inneholdt.

Hvordan:

Vi valgte ut 5 issues fra backloggen, alle formulert med utgangspunkt i produktkravene til prosjektet. Først gikk vi gjennom hvert issue og tagget de med taggene Maintainability, Security, Reliability, Usability, Compatibility, Performance og Portability.

Deretter limte vi hvert issue inn i QualityTagger, noterte verktøyets forslag og diskuterte og sammenlignet dem med våre egne tagger. Underveis oppdaget vi at issueene burde være på engelsk for at QualityTagger skulle fungere, og vi oversatte dem før vi fortsatte. Forslagene ble svært forskjellige på norsk vs engelsk, vi opplevde de engelske som mer fornuftige.

Refleksjoner:

Positivt: Aktiviteten skapte nyttig diskusjon. Det var interessant å se at verktøyet valgte andre tagger enn oss, og begrunnelsen fikk oss til å se noen av issueene fra en ny vinkel. PROD-02 håndtere nettverksfeil og API-feil er et eksempel på dette. Vi tagget den med Reliability, Performance og Portability, mens QualityTagger valgte Usability. Begrunnelsen var at tydelige feilmeldinger handler om at brukeren forstår hva som skjer og kan handle deretter. Våre tagger hadde systemet i fokus, men QualityTagger hadde brukeren.

Negativt: Vi opplevde at verktøyet hadde begrenset nytte for vår del. Fra backlog refinmenten hadde vi allerede kategorisert backloggen på en måte som ga oss mer relevant informasjon enn kvalitetstaggene. Siden vi brukte issues formulert direkte fra produktkravene ga det lite mening å reformulere de, de er tross alt allerede presist formulert med utgangspunkt i et formelt dokument. Vi kan likevel se nytten av omskrivningsfunksjonen i andre situasjoner.

Når vi testet den med en av de mindre presise backloggene knyttet til vår vinkling av prosjektet ble det tydelig at QualityTagger ikke hadde tilstrekkelig med kontekst, da klassifiseringen ga lite mening. En forbedring ville vært om den ikke bare forklarte hvorfor en tag passet best, men også hvorfor de andre taggene passer mindre. Dette ville gjort diskusjonen langt lettere.

Vi så nytten av en form for kvalitetstagger i større prosjekter med en mer omfattende backlogg og større teams, men vurderte det som lite hensiktsmessig å bruke aktivt videre i dette prosjektet.

Bilde/dokumentasjon

Svaret er levert

Leveret: 13. april 2026 14:18

IN2000 V26 Activity: Enriching the backlog with quality tags

Du vil motta en e-post med lenke til svaret.

ID	Issue	Våre tags	QualityTagger sine
CASE-01a	Integrasjon mot Frost API via IFIs proxyserver. API-kall mot Frost skal gjøres asynkront og gå via IFIs proxyserver i stedet for direkte til frost.met.no.	Compatibility Security	Compatibility
PROD-02	Håndter nettverksfeil og API-feil. Appen skal ikke krasje ved manglende internett eller mislykkede API-kall → heller varsle brukeren med en tydelig feilmelding.	Reliability Performance Portability	Usability
PROD-03	Støtt Activity Lifecycle. Viktig tilstand som valgt lokasjon skal ikke forsvinne dersom appen avbrytes, enheten roteres eller brukeren navigerer til andre skjermer.	Portability Usability Reliability	Reliability
PROD-06	Universell utforming WCAG 2.1. Appen bør ta hensyn til WCAG 2.1-standarden og bruke anbefalte tilgjengelighetsverktøy i Jetpack Compose. Layout skal fungere godt uavhengig av skjermstørrelse og skjermopløsning.	Usability Portability Maintainability (indirekte)	Compatibility
PROD-04	Minimum 10 enhetstester. Det skal skrives automatiske enhetstester for sentral funksjonalitet i appen, med minst 10 tester totalt.	Maintainability Reliability Performance?	Reliability

Veiledermøte 13.april 3/4 1p

Hvem: Hele teamet

Dato: 13.april

Vi likte godt å ha veiledning hver uke da det ga oss kontinuerlig og jevn mulighet for å stille spørsmål og få tilbakemelding.

Temaer og avklaringer knyttet til møtet:

- Viste fram Arkitekturskissen
 - fikk tilbakemelding på feil bruk av pilene
 - Avklaring på at vi ikke trengte å bruke mermaid
- Presenterte backlogge refinementen
 - fikk tilbakemelding på kravene
 - Veiledet os til å ikke inkludere et kartlag med geoscore
- Presenterte appen så langt
 - Vi var på riktig spor
- Presenterte tankene om hvordan vi skal implementere algoritmen
 - De mente algoritmen var planlagt bra
 - Fikk forslag til å bruke AI til å generere rapport basert algoritme indeksene

Refleksjoner:

Vi opplevde det som utfordrende å formulere algoritmen, men møtet ga oss verdifull avklaring om at tilnærmingen vi hadde valgt var ikke bare tilstrekkelig, men faktisk god.

En særlig positiv erfaring var den konstruktive tilbakemeldingen vi fikk på arkitekturskissen. Vi hadde anstrengt oss og slo oss til ro med en dårlig Mermaid generert skisse, fordi vi trodde det var et krav. Men veilederne avklarte at det ikke var tilfellet. Veilederne påpekte også at vi hadde brukt pilene feil.

Uke 17

Datainnsamling: geriljatesting 2p

Hvem: Veronica

Dato: 23.april og 24.april

Geriljatesting gjorde vi på dette tidspunktet da utviklingen i Figma var kommet såpass langt i prototypene og hensiktsmessig å gjøre før implementering i appen.

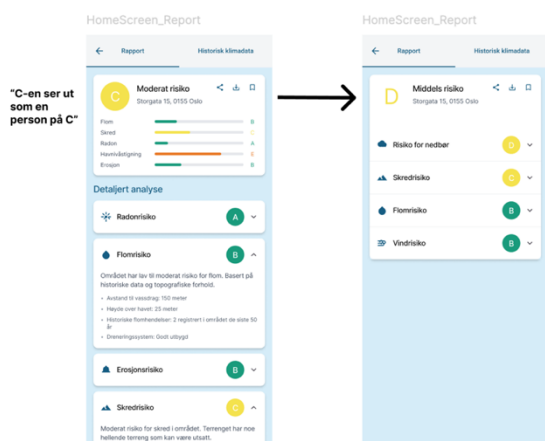
Hvordan:

Testingen ble gjort ved å vise frem prototypen i designverktøyet Figma sin app på mobil der brukerne fikk interagere med den direkte. Det ble test ved hjelp av metoden «Think aloud» og observasjon, samt et par konkrete oppgaver brukerne ble bedt om å gjøre. Det ble tatt skjermopptak med lyd etter samtykke. En test-guide ble brukt som støtte i gjennomførelse. Det ble testet med totalt fem brukere.

Refleksjoner:

Dette erfarte vi som en rask, enkel og nyttig testing uten høy grad av innsats. Det var enkelt å finne brukere og krevde lite planlegging da testene tok kort tid. Vi erfarte det som nyttig når man ønsker å hente inn i større grad kvantitativ data, fremfor kvalitativ. Vi fikk avdekket funn både for UI, tekstlige formuleringer og nye funksjoner. Eksempelvis syntes flere av brukerne at C-en i rapporten til et sted så ut som en persons profilbilde med initiale. Sliderne ga også heller ikke mening for flere. Vi opplevde det som både gøy og motiverende, og kunne tenkt oss å gjøre det igjen ved eventuell fortsettelse av prosjektet.

Bilde/dokumentasjon



Par-programmering 1p

Hvem: Veronica og Matilda

Dato: 24 april

Vi valgte å gjennomføre parprogrammeringen på dette tidspunktet da det var en fin og avgrenset oppgave som passet godt til parprogrammering. Designeren hadde også uttrykt at hun gjerne ville være med på å jobbe med API-er, da dette er relevant for eksamen og viktig for god læring på tvers av roller i teamet

Hvordan:

Parprogrammeringen ble en halvveis refaktorering der vi gikk fra Frost V1 tilbake igjen til Frost V0, som hadde bedre støtte for historiske klimanormaler.

Vi startet med å gå gjennom Frost sine 700 forskjellige element id-er for å danne oss et overblikk over hva de forskjellige elementene faktisk forteller. Med utgangspunkt i casbeskrivelsen og diskusjoner om hva vi selv ville være interessert i å vite før vi kjøpte bolig et sted, landet vi på å vise historisk klimadata for temperatur, nedbør, snø, vind og solskinn.

For å hente data fra V0 implementerte vi hjelpemetoden `findNearestV0Sources`. Denne tar inn koordinater og antall stasjoner og returnerer de nærmeste meteorologiske stasjonene. Vi fant og implementerte element-id-er for følgende. Tre forskjellige for temperatur. En generell en, en for gjennomsnittet av de varmeste og kaldeste dagene. For solskinn antall timer sol på sensoren per måned `sum(duration_of_sunshine P1M)`. For nedbør ble det antall dager med nedbør og gjennomsnittlig snødybde. Historisk data for vind fikk et skjelett og implementasjonen kom senere.

Basert på erfaringen fra mobbprogrammeringen valgte vi å ikke bruke timer for å bytte roller, men byttet heller der det falt naturlig. I praksis hvert 10-15 minutt. Selve parprogrammeringen tok ca 2 timer.

Refleksjoner:

Det var lærerikt å måtte forklare for en som ikke har jobbet så mye med API-laget. Veronica har hatt hovedansvar for design i prosjektet, og det å forklare og bli forklart førte til bedre forståelse for begge. Å sette ord på tekniske valg tvinger frem en mer bevisst forståelse av hva man egentlig har gjort som var positivt.

En negativ ting er at parprogrammeringen skjedde midt i en refaktorering. Dette gjorde det litt vanskelig å se det store bildet underveis. Vi skulle gjerne også sett dataen visuelt, men det ble ikke tid til UI-integrering i løpet av økten.

Par programmering var en positiv opplevelse og det er noe vi gjerne gjør igjen.

Lenke til commit på github:

<https://github.uio.no/IN2000-V26/team-20/commit/019441ef96d091dc0bf7253e71426437f635d9b3>

Bilde/dokumentasjon



Uke 18

Andre retrospektiv (obligatorisk) 2p

Hvem: Hele teamet

Dato: 27.april

Hvorfor nå?

På dette tidspunktet følte vi både behovet for en retrospektiv, det var gått noen uker siden forrige, og samtidig fortsatt tid og mulighet til å gjøre endringer for siste fasen av prosjektet.

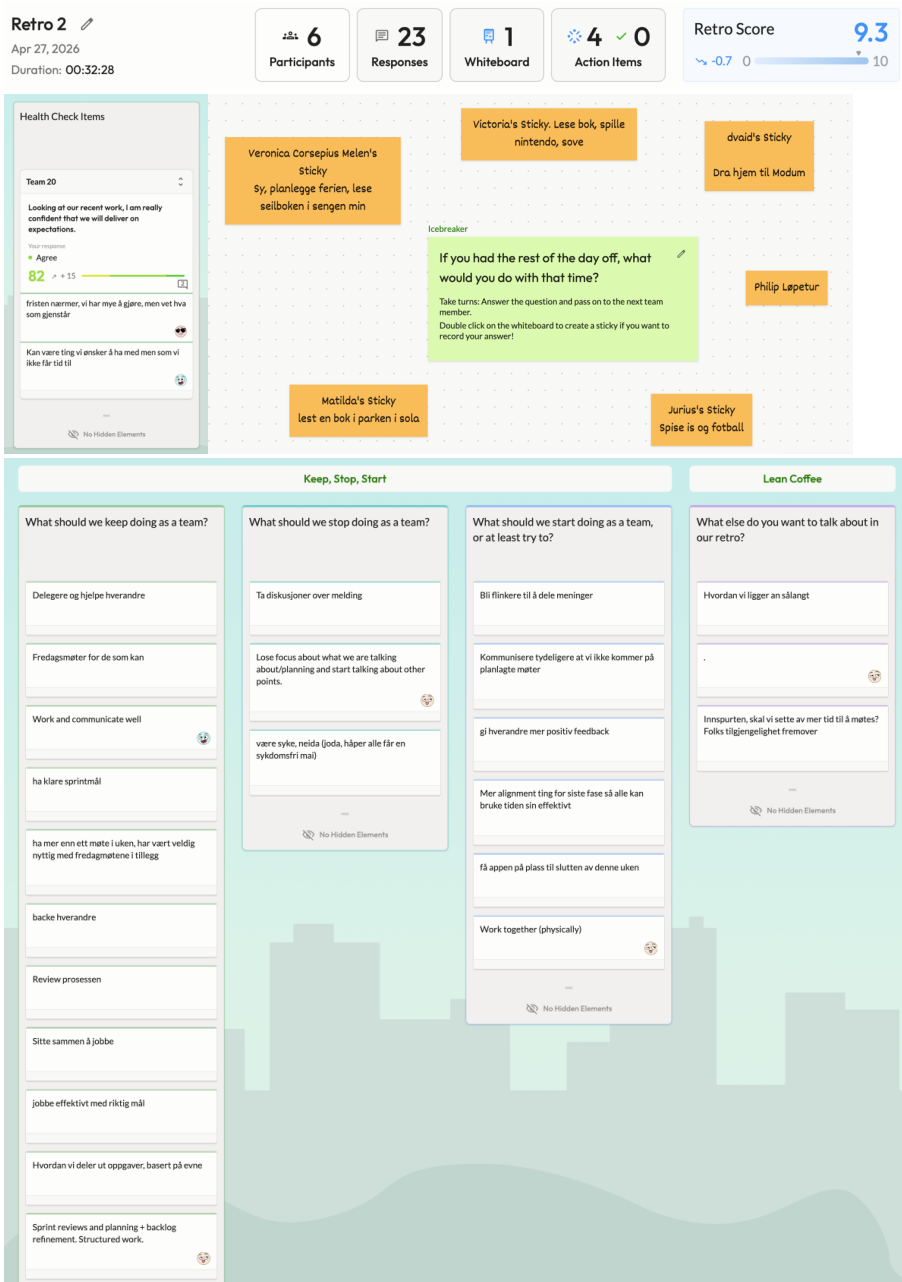
Hvordan

Etter god erfaring med bruk av Echometer, ønsket vi å gjenta suksessen og tok med oss de delene vi syntes fungerte best. Det ble en litt kortere versjon etter lærdom fra sist. Vi gjentok bruken av survey i forkant, «icebreaker», et av de samme «Health Check Items» for enkel sammenlikning av resultat, «Keep, Stop, Start» og «Lean Coffee». Vi satte av 1t til aktiviteten i plenum, som sist, men var fornøyde etter en drøy halvtime denne gangen.

Refleksjoner

Vi erfarte det som svært nyttig i det stadiet vi var ettersom vi hadde litt tendenser til konflikt uken før, der dette ble en fin måte og anledning til å få snakket samlet om det, samt oppklare uklarheter og misforståelser. Retroen ble denne gangen anonymt scoret til 9,3/10, en liten nedgang fra sist. Muligens fordi denne kan tenkes å ha vært noe mer ubehagelig enn den forrige med flere konkrete punkter for samtalen. Dette anser vi som en desto viktigere grunn til å gjennomføre slike aktiviteter, og ville gjort det igjen ved eventuell fortsettelse av prosjektet.

Bilde/dokumentasjon



Datainnsamling: brukertesting (obligatorisk) 2p

Hvem: Veronica og Matilda

Dato: 1.mai og 2.mai

Hvorfor nå?

Appen hadde begynt å ta ordentlig form, med tiltenkt bruker-flow. Vi ville se hvordan flowen fungerte for brukere og eventuelt kartlegge forbedringspunkter for bedre brukeropplevelse.

Hvordan

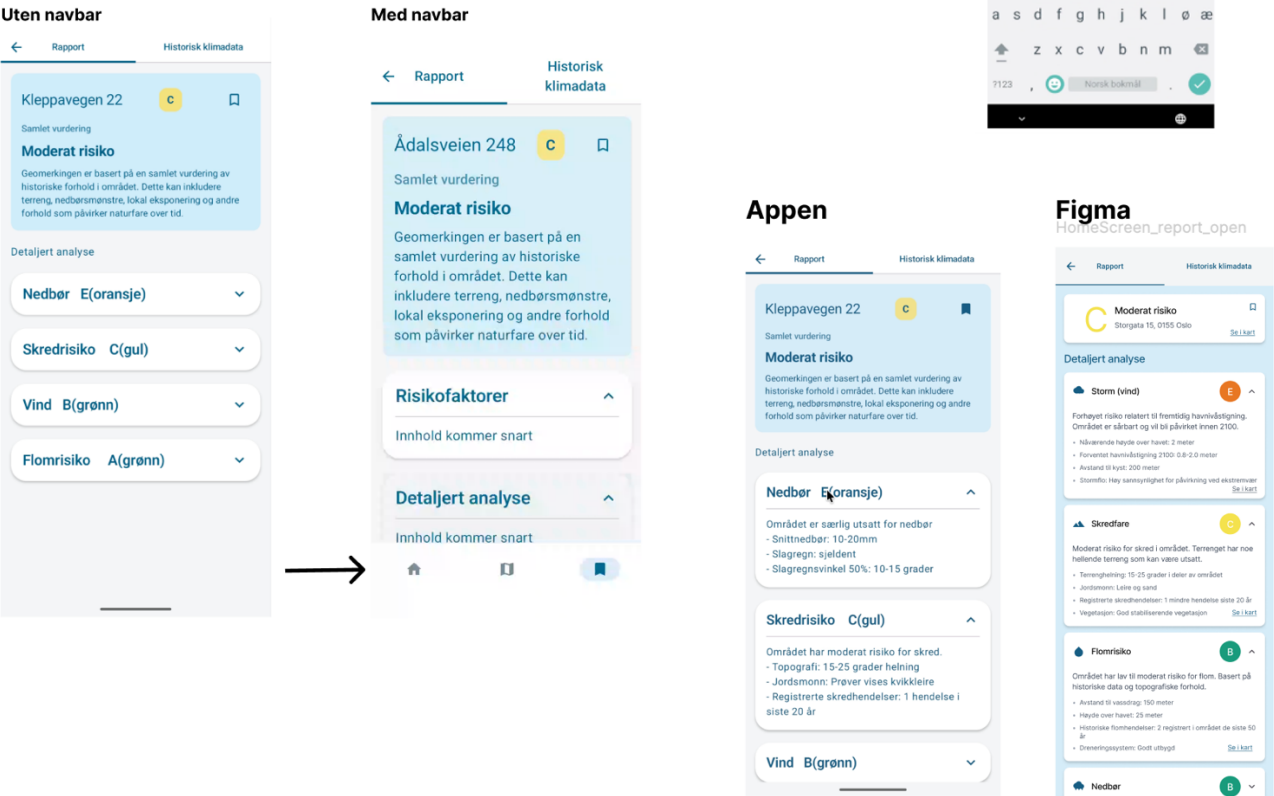
Vi testet med totalt tre brukere. Vi testet primært appen, men supplerte med Figma som på enkelte områder hadde mer detaljer på tiltenkt utseende. Eksempel rakk vi ikke å implementere detalj-bokstavene med farge og hardkodet «(oransje» ved siden av, så dette ble presentert med Figma. Brukerne fikk en introduksjon der vi gjorde det tydelig at det ikke er dem vi tester, men selve appen og at deres meninger er verdifulle, både positive og negative, og tas imot med takknemlighet og ikke kritikk. I testen benyttet vi oss av observasjon og «Think aloud», der brukeren ble bedt om å tenke høyt slik at vi fikk bli med på deres forståelse av appen. Det ble tatt skjermpoptak med lyd etter samtykke. De to første brukerne testet appen i emulator, mens siste fikk testet på sin egen android-mobil. For å inkludere flere i teamet i innsiktsarbeidet, ikke bare UX-designeren, valgte vi å ha med en av våre utviklere på en av brukertestene.

Refleksjoner

Brukertestene ga svært nyttig innsikt i brukernes forståelse av appen, dens intuitivitet og brukeropplevelse. Av sentrale funn fant vi ut av at vi skulle ha navigasjonsbaren på alle screens, både for enklere navigasjon i sin helhet uavhengig av hvor i appen brukerne var, i tillegg til å lettere forstå hvor rapportene faktisk blir lagret. Dette var svært tydelig fra de to første testene, at vi vurderte det som mest nyttig å endre allerede før den tredje brukertesten, slik at vi fikk testet bruken med navigasjonsbar på alle screens også. Der vi blant annet oppdaget en bug som markerte saved-knappen i navigasjonsbaren selvom det ikke var der brukeren kom fra. Vi erfarte det som svært nyttig å få testet appen på en fysisk enhet da dette avdekket flere funn som måtte fikses, som for eksempel at skrift mistet synlighet. Vi sorterte funnene i «must have» og «nice to have».

Valget om å ha med en utvikler erfarte vi som lurt da det ble enklere og raskere tverrfaglig samarbeid med funnene i etterkant med felles forståelse.

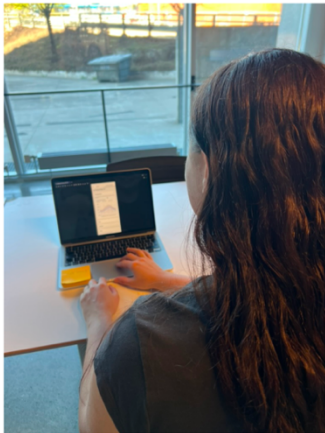
Bilde/dokumentasjon



Bruker 1



Bruker 2



Bruker 3



Uke 19

Veiledermøte 4.mai 4/4 1p

Hvem: David, Jurius, Matilda, Veronica, Victoria + veiledere

Dato: 4. mai

Fast ukentlig veiledermøte. Vi viste frem appen slik den ser ut nå og hadde et par praktiske spørsmål vi ønsket avklaring rundt.

Temaer og avklaringer fra møtet:

- Viste frem appen.
 - o Noen mindre UI-justeringer. En knapp er for nærme en annen en, det kommer ikke tydelig nok frem hva måleenheten her er.
 - o Caching-strategi: cache værdata og målestasjoner i minnet, evt i Room. Met oppfordrer til færrest mulige API-kall.
- Mulig å få veiledning til presentasjonen i uke 21. Dette blir dessverre ikke aktuelt for teamet grunnet andre eksamener.
- Spørsmål rundt hvordan vi skal håndtere API-nøkler når vi leverer. Veileder skal undersøke.
- Tips til prosesslogg: noter 3-4 action points per veiledermøte

Vi var i siste fase av prosjektet og begynte å nærme oss innlevering. Vi hadde lyst til å treffe med de ulike aspektene av innleveringen vår så vi hadde mye nytte av å snakke med veileder.

Uke 20

Siste veiledermøte 12.mai 5/4

Hvem: David, Jurius, Matilda, Veronica, Victoria + veiledere

Dato: 12. mai

Siste ukentlige veiledermøte før levering av prosjektet. Vi viste frem appen slik den ser ut nå, gikk gjennom rapporten og deler av prosessloggen

Temaer og avklaringer fra møtet:

- Viste frem appen:
 - o Beskrivelse på grafene, veilederne liker lengden, men vi bør se litt over formuleringen av de
 - o «Øyet» i kartskjermen er ikke veldig tydelig hva den gjør. Bør vurdere gjøre det tydeligere at denne knappen vil zoome inn til korrekt kartzoom for å vise kartoverlayet
 - o Scoren virker mer rettferdig, bra vi gjorde justeringer på terskelverdiene for scoren
- Gikk gjennom rapporten:
 - o Oppklaring rundt testverktøy i prosessdokumentasjonen
 - o Strukturer vedleggene slik at om du i teksten trykker på «Vedlegg 3, Kravspesifikasjoner» blir du automatisk navigert til riktig vedlegg i vedlegglisen.
 - o David la inn et issue på google maps sin github, det ble fikset i den nyeste versjonen deres → kult! Få det med i rapporten
- Oppklaring fra forrige veiledermøte: hardkode API-nøklene slik at sensor kan kjøre appen direkte fra levert zip-fil uten ekstra konfigurering. Dette vil bety at vi sjekker nøklene inn i git. Ikke ideelt, men greit her siden de bare gir tilgang til ufarlige ressurser.

Siste veiledning før prosjektinnlevering var oppklarende og en fin avslutning på prosessen. Vi fikk flere gode og konkrete tips om hva som skal inkluderes hvor i rapporten. Veileder ga oss masse motivasjon. Vi har jobb igjen, men nok tid til at vi kommer godt i mål!